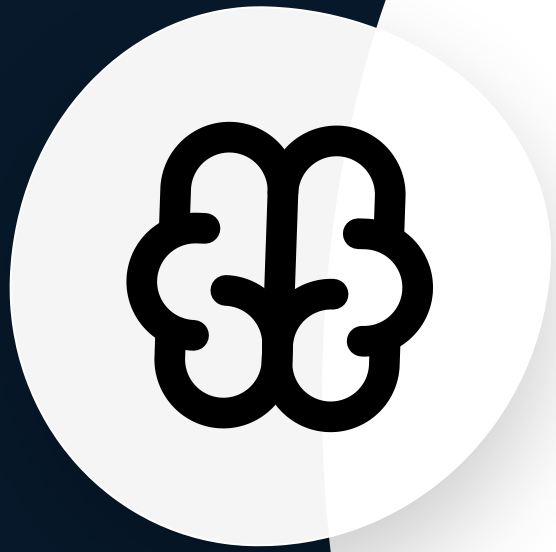
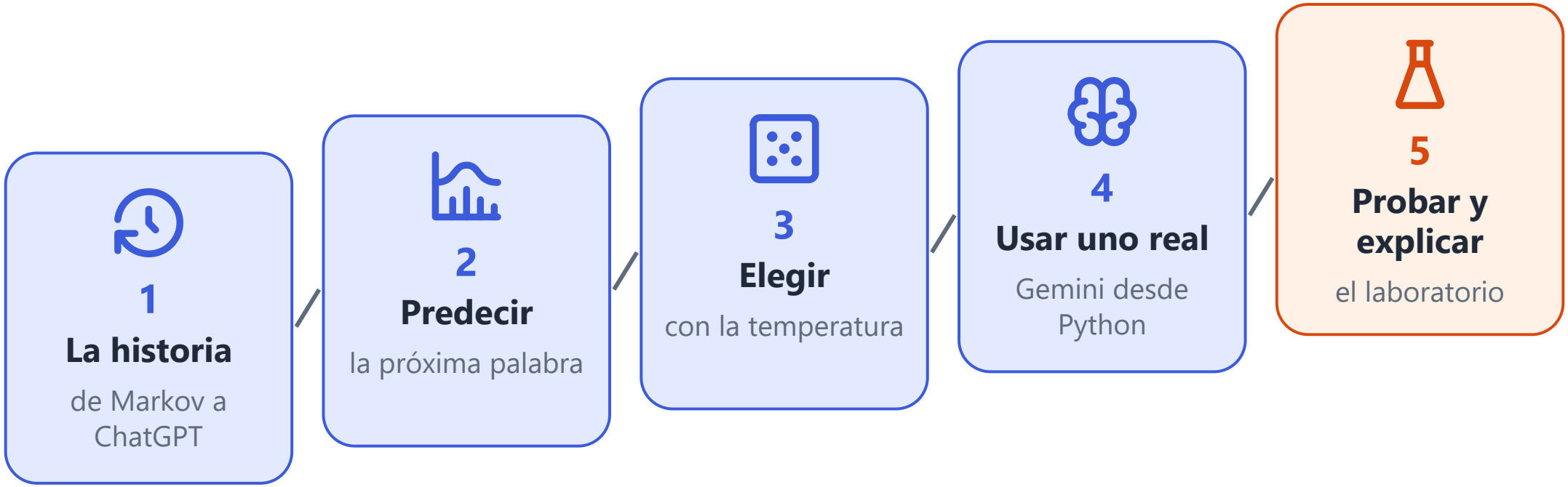
Qué es un LLM

M1: Fundamentos de LLMs



Hoy subimos cinco pasos, sin saltos



Arrancamos de lo que ya sabés de ML.

Predecí: ¿cómo sigue esta frase?

«La capital de Uruguay es ? »



¿Busca?



¿Entiende?



¿Predice?

Anotá tu apuesta y qué probabilidad le das a «Montevideo».

Posibles respuestas: busca, entiende, predice



Busca

no

No tiene buscador ni internet adentro.



Entiende

discutible

No sabe qué es Uruguay como vos.



Predice

sí

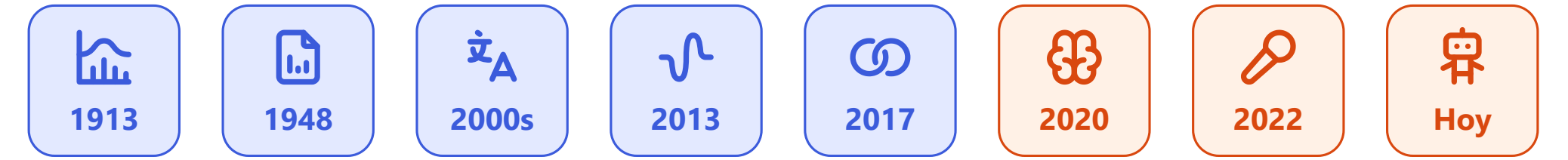
Elige el próximo pedazo más probable.

Lo medimos en un rato.

Cien años prediciendo lo que sigue

CONTAR LO QUE SIGUE

MODELOS GRANDES



Markov
qué sigue
a cada vocal

Shannon
frases
contando palabras

El celular
sugiere
la próxima

word2vec
cada palabra,
un vector

Transformer
mira todo
el texto

GPT-3
mucho
más grande

ChatGPT
ajustado
para conversar

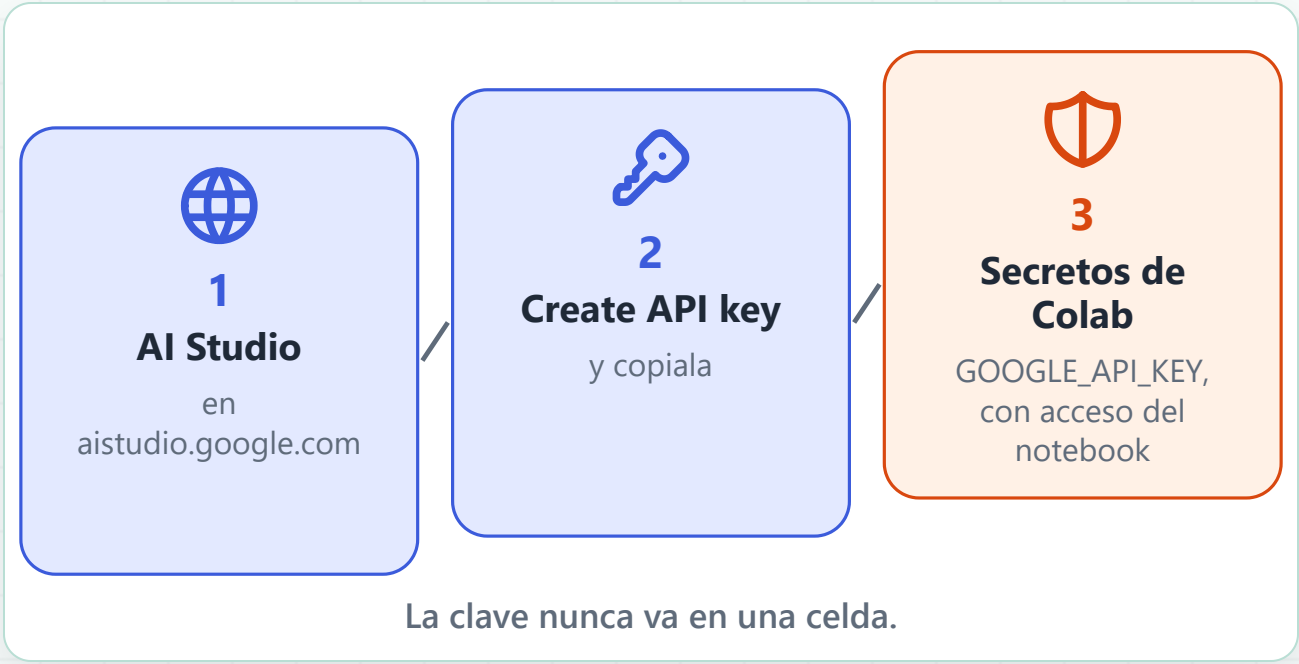
Agentes
buscan y
usan herramientas

La idea nunca cambió: mirar lo anterior y apostar por lo que sigue.

Antes del cuaderno: tu clave de Gemini

Objetivo: que el cuaderno pueda hablar con Gemini.

1. aistudio.google.com/apikey → **Create API key**.
2. Colab, llave de la izquierda (**Secretos**) → `GOOGLE_API_KEY`, con **Acceso del notebook**.



Cuaderno, parte 1: Python hablando con Gemini

Objetivo: repasar Python mientras le hablás a Gemini.

- `m1_python_de_cero.ipynb` : en juanfkurucz.com/ucu-la, práctica M1 → «Open in Colab».
- Ejercicios 1 a 5 (10 min); el resto, en casa.



Qué deberías ver:

`Montevideo` en el 5.

Paso 2 de 5: Predecir

Cada ejercicio te dice si salió y, si no, una pista

CO m1_python_de_cero.ipynb

```
tema = "un token"   # cuando te salga, probá con otro tema
pregunta = None  # ← tu respuesta
print(pregunta)
revisar("Ejercicio 1", pregunta, f"Explicá en una oración qué es {tema}.",
        'pregunta = f"Explicá en una oración qué es {tema}."')

if pregunta:
    r = modelo.invoke(pregunta)
    print(r.text)
```

```
None
❌ Ejercicio 1: obtuviste None, se esperaba 'Explicá en una oración qué es un token.'
   Pista: pregunta = f"Explicá en una oración qué es {tema}."
```

¿Qué es un LLM?

Le das un texto
«La capital de Uruguay es»

→

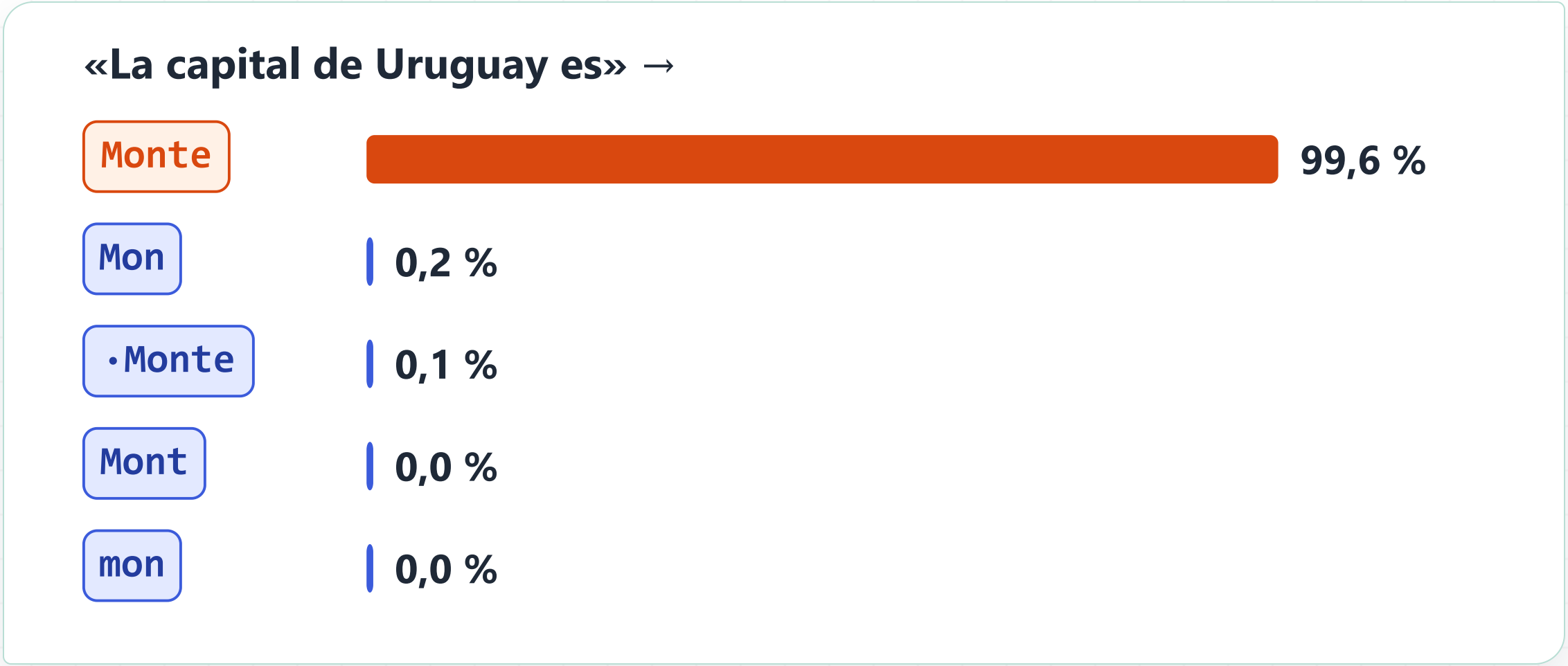
Te da lo que sigue
«Montevideo», con 99,6 %

- **LLM** = un modelo que predice el próximo pedazo de texto.
- Como el teclado del celular, entrenado con casi todo internet.

Un LLM es un clasificador que ya conocés

	Clasificador de ML	LLM
Entrada	Features	Todo el texto anterior
Clases	Pocas	100 a 260 mil pedazos de palabra (tokens)
Salida	Softmax	Softmax
Etiqueta	La anota alguien	La palabra que realmente seguía

Cada pedazo posible tiene su probabilidad



Ni busca ni entiende: predice.

Generar es clasificar, agregar y repetir



Seis vueltas: elige, agrega y repite

1	El fin de semana fuimos a la	`play`	72 %
2	El fin de semana fuimos a la	`play` `a`	100 %
3	El fin de semana fuimos a la	`play` `a` `·y`	91 %
4	El fin de semana fuimos a la	`play` `a` `·y` `·disfrut`	91 %
5	El fin de semana fuimos a la	`play` `a` `·y` `·disfrut` `amos`	100 %
6	El fin de semana fuimos a la	`play` `a` `·y` `·disfrut` `amos` `·del`	83 %

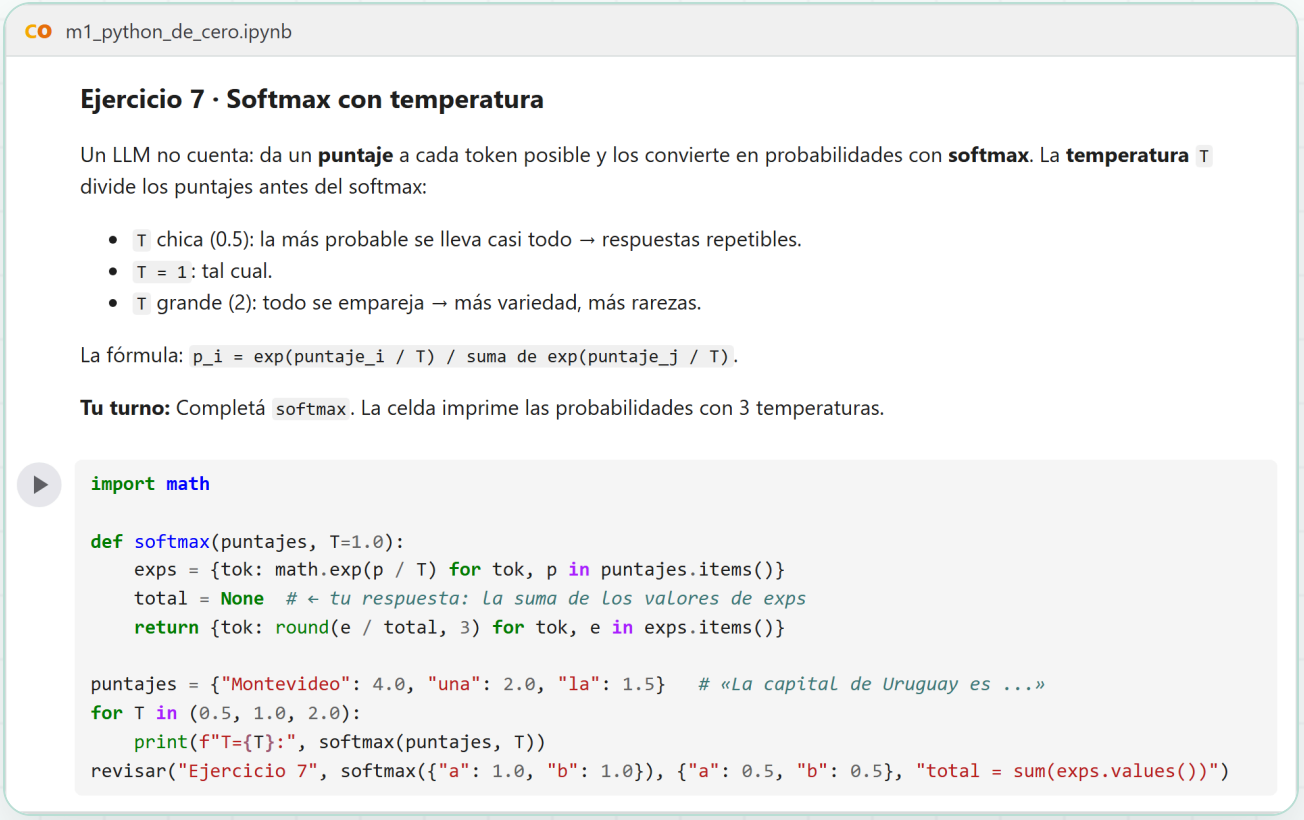
Gana siempre la primera.

Cuaderno, parte 2: qué palabra sigue

Objetivo: armar un modelo de lenguaje de juguete.

Ejercicios 6 y 7, 5 min: la tabla de qué palabra sigue y el softmax con temperatura.

Es lo de 1913 y 1948, hecho por vos.



CO m1_python_de_cero.ipynb

Ejercicio 7 · Softmax con temperatura

Un LLM no cuenta: da un **puntaje** a cada token posible y los convierte en probabilidades con **softmax**. La **temperatura** `T` divide los puntajes antes del softmax:

- `T` chica (0.5): la más probable se lleva casi todo → respuestas repetibles.
- `T = 1`: tal cual.
- `T` grande (2): todo se empareja → más variedad, más rarezas.

La fórmula: `p_i = exp(puntaje_i / T) / suma de exp(puntaje_j / T)`.

Tu turno: Completá `softmax`. La celda imprime las probabilidades con 3 temperaturas.

```
import math

def softmax(puntajes, T=1.0):
    exps = {tok: math.exp(p / T) for tok, p in puntajes.items()}
    total = None  # ← tu respuesta: la suma de los valores de exps
    return {tok: round(e / total, 3) for tok, e in exps.items()}

puntajes = {"Montevideo": 4.0, "una": 2.0, "la": 1.5}   # «La capital de Uruguay es ...»
for T in (0.5, 1.0, 2.0):
    print(f"T={T}:", softmax(puntajes, T))
revisar("Ejercicio 7", softmax({"a": 1.0, "b": 1.0}), {"a": 0.5, "b": 0.5}, "total = sum(exps.values())")
```

La temperatura cambia la variedad

Temperatura (T): un número que estira o aplana las probabilidades antes de sortear.

1. Lo que da el modelo



playa 75 %

costa, montaña, campo, ciudad

2. T baja: se estira



playa 97 %

costa, montaña, campo, ciudad

2. T alta: se aplana



playa 48 %

costa, montaña, campo, ciudad

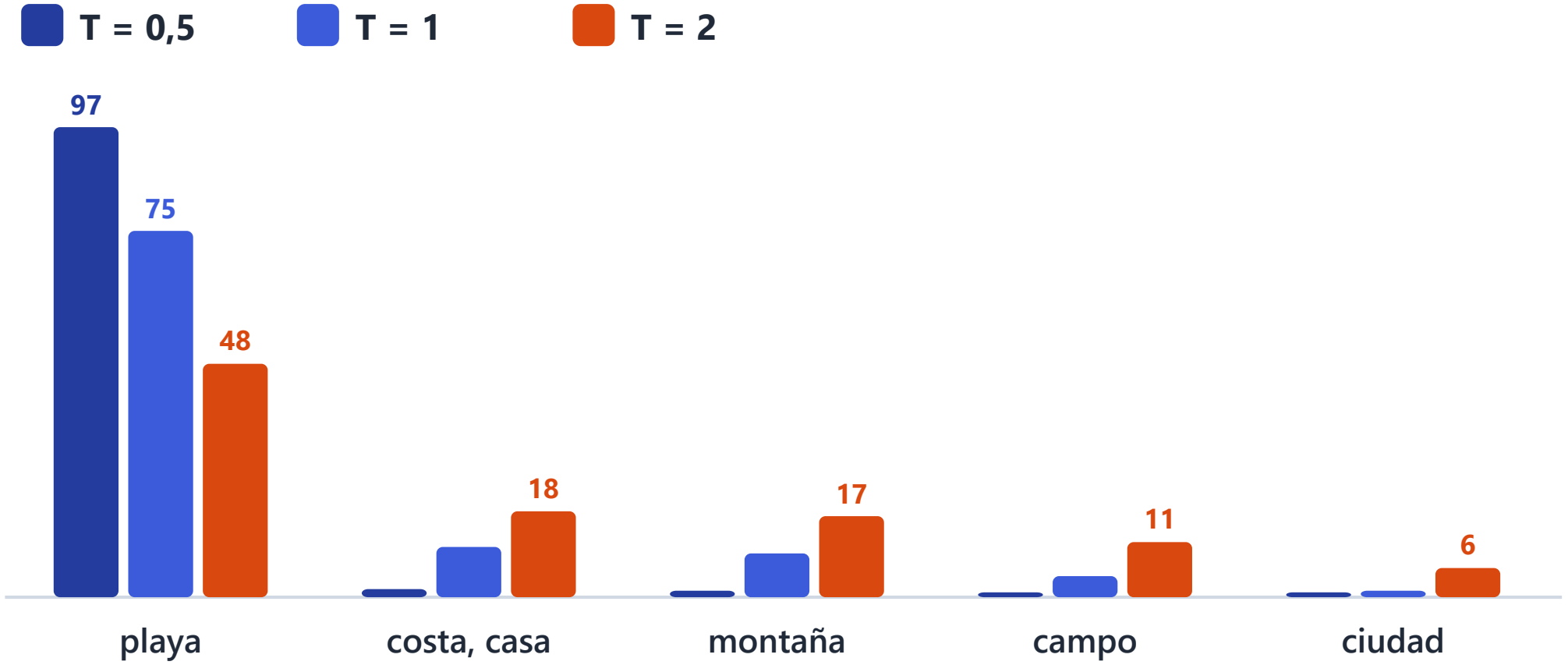
3. Sortear

una palabra con esas probabilidades

Sortear: un dado cargado. Con playa al 75 %, sale playa unas 75 de cada 100 veces.

Cambia la variedad, no la verdad.

La temperatura estira o aplana lo mismo



Mismas opciones, otro reparto.

Gemini 3.5 ignora la temperatura

respuestas distintas de 5, por corrida

Gemini 3.5 **T = 0**



Café Pocitos, MonteCafé, Montevideana…

Gemini 3.5 **T = 1**



Gemini 3.1 **T = 0**



«Montevideo Mío» las 5 veces, en las 3 corridas

Gemini 3.1 **T = 1**

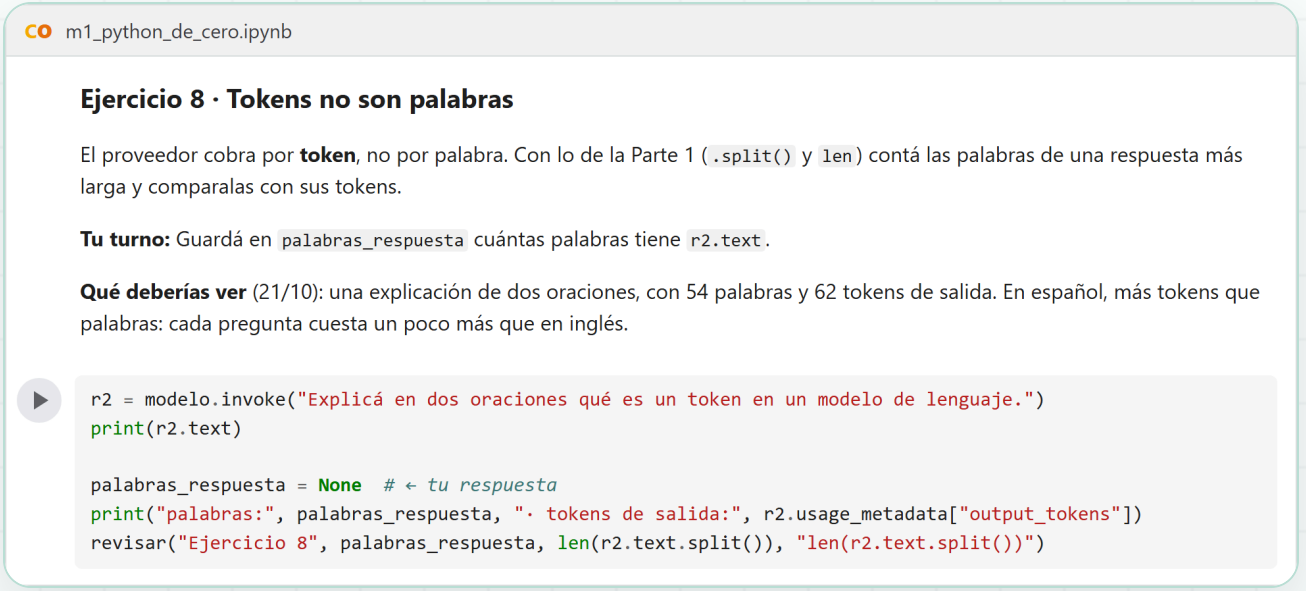


Café del Sur, Montevideo Místico…

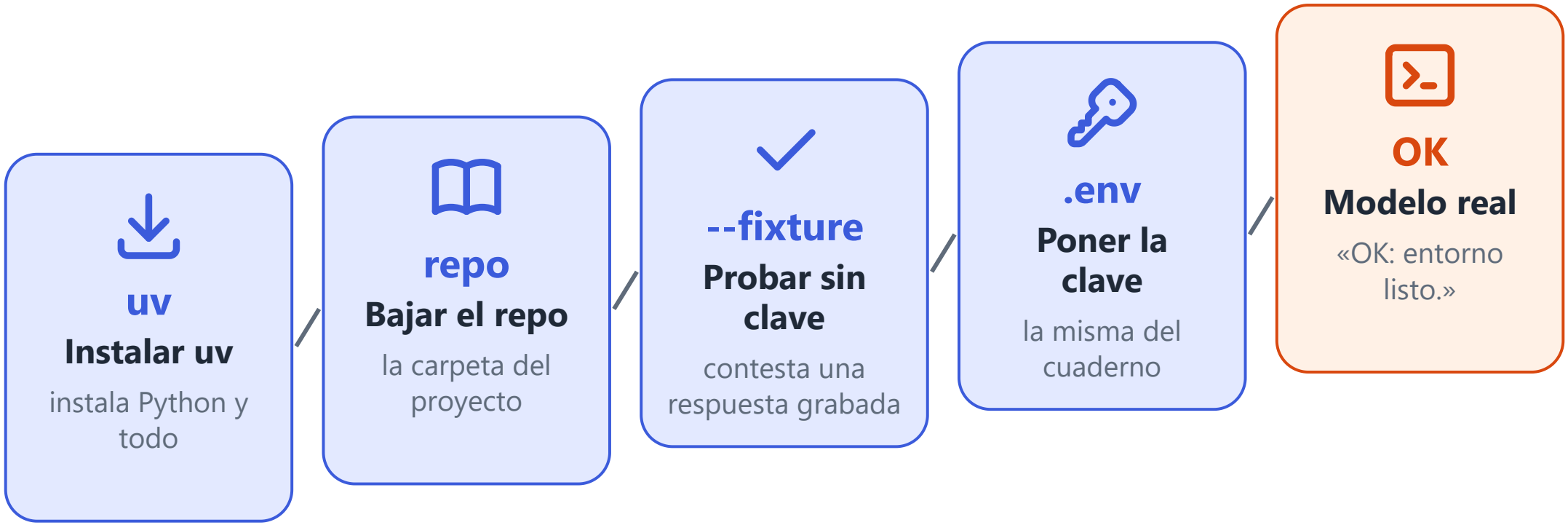
Cuaderno, parte 3: tokens, temperatura y JSON

Objetivo: ver de cerca tokens y temperatura en un LLM real.

- Ejercicios 8 y 9, 15 min: tokens contra palabras y temperatura.
- El 10 (JSON), en casa.



Entorno guiado: lo mismo, desde tu computadora



Si check_setup.py dice «OK: entorno listo.», estás arriba.

Paso a paso: «Antes de la clase 1», en el aula virtual.

El modelo no ve palabras: ve tokens

Castellano: 11 tokens

`Los` `·estudiantes` `·descon` `fi` `aban` `·de` `·las` `·respuestas` `·del` `·modelo` `.`

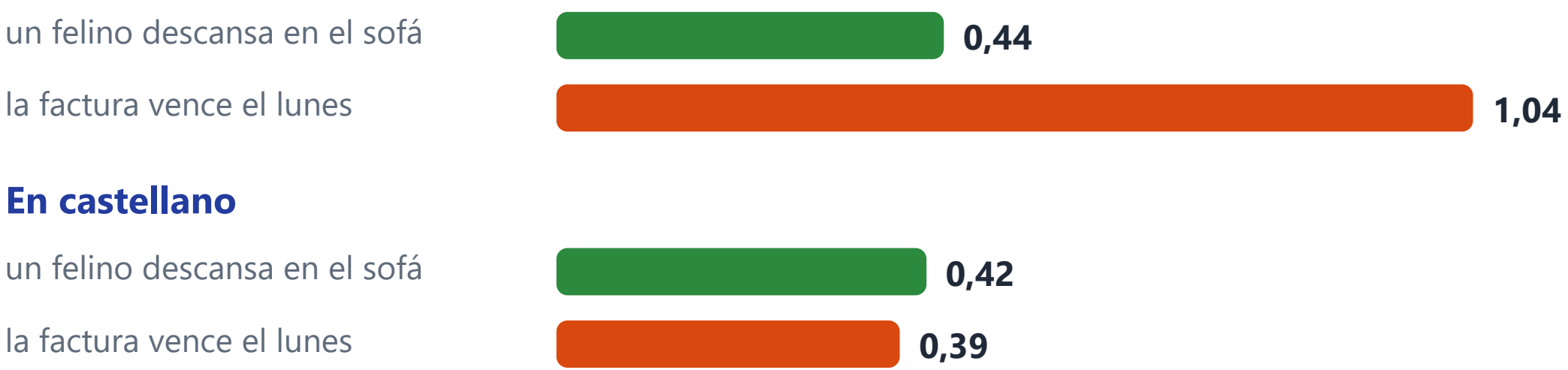
Inglés: 9 tokens

`The` `·students` `·distr` `usted` `·the` `·model` `'s` `·answers` `.`

Token = un pedazo de palabra. Se paga por token.

Lo parecido queda cerca, si el modelo sabe el idioma

Distancia desde «el gato duerme en el sillón» (0 = igual)



En castellano, la factura queda tan cerca como el felino: este modelo no separa.

Texto = vector (**embedding**).

La atención decide qué palabras importan

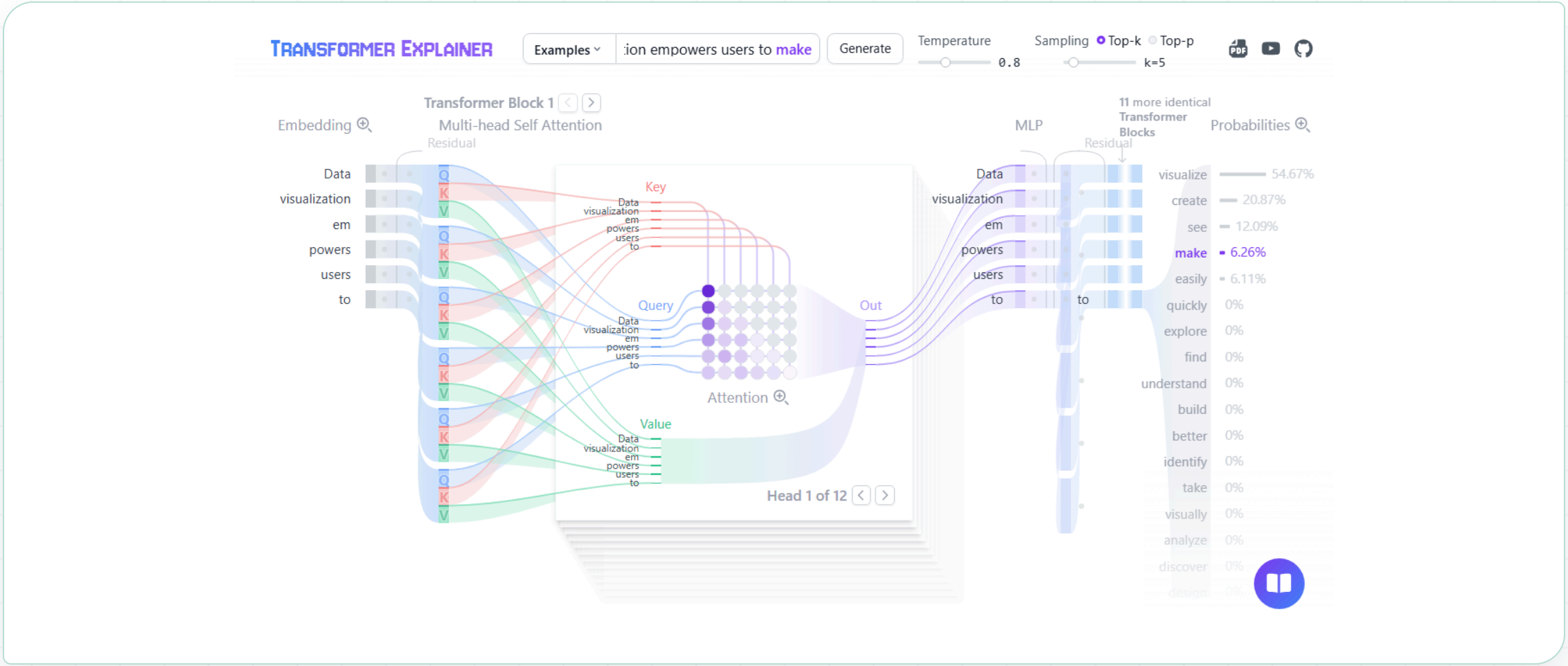
Texto
«Me senté en el banco de la plaza y estaba…»

→

Tiene que pesar más
«plaza» y «senté»: es un asiento, no un banco de dinero

- Lo agregó el Transformer (2017).
- Cómo se ve por dentro, en la próxima.

GPT-2 por dentro: sorteó «make», con 6 %



Sorteó: el 6 % sale 1 de 16. Transformer Explainer

Predice lo probable, no lo verdadero



Por eso inventa con seguridad, sobre todo nombres, fechas, cifras y citas.

Le preguntamos por un libro que no existe

Gemini 3.5 Flash-Lite

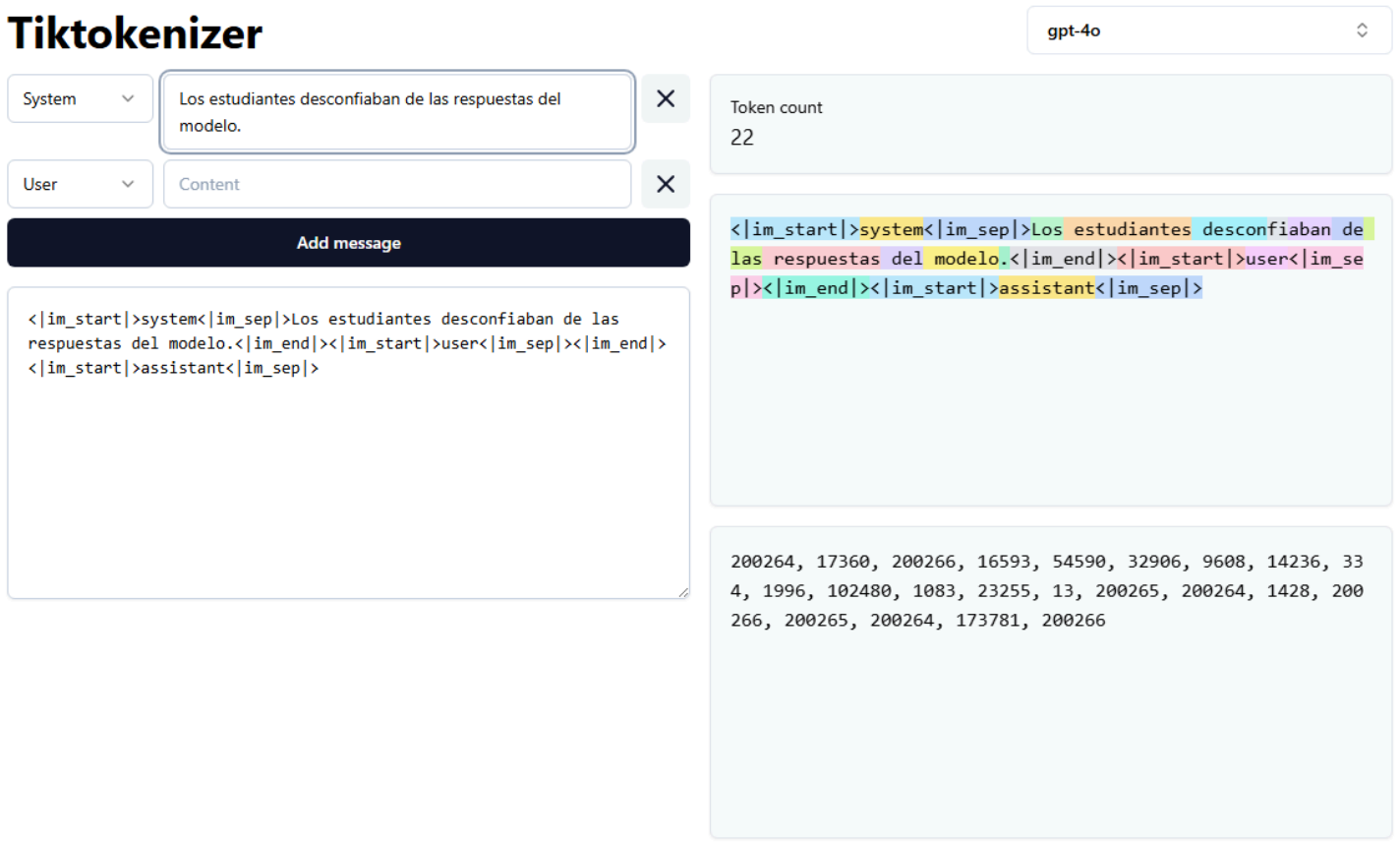
Resumí en dos frases el libro «El álgebra de los ríos» de Mario Benedetti.

«El álgebra de los ríos», de Mario Benedetti, es un poemario de tintes melancólicos donde el autor reflexiona sobre el paso inexorable del tiempo, la memoria y la soledad…

Ese libro no existe

Predijo lo probable, con seguridad.

En Tiktokenizer: 11 tokens de la frase y 11 del chat



Probalo: Tiktokenizer

Paso 5 de 5: Probar y explicar

El lab: modelo, prompt, temperatura y repeticiones

CO m1_laboratorio.ipynb

2 · Perillas

- `MODELO_T`: la temperatura la usa Gemini 3.1 Flash-Lite; el 3.5 la ignora.
- Tope: 20 llamadas por persona (`TOPE`). Cada repetición es una llamada.

```
MODELO_T = "google_genai:gemini-3.1-flash-lite"
PROMPT = "Inventá un nombre para una cafetería en Montevideo. Respondé solo el nombre."
TEMPERATURA = 0.0
REPETICIONES = 5
TOPE = 20
```

3 · Experimento 1: temperatura

Predecí antes: con 0, ¿cuántas distintas sobre 5? Corré, anotá, cambiá `TEMPERATURA` a 1 en las perillas y corré de nuevo esa celda y la de `correr(...)`.

En ucu-la, práctica M1 → «Open in Colab».

Laboratorio: cambiá una cosa y mirá

Objetivo: cambiar una sola cosa y explicar lo que pasó. Elegí un experimento (práctica M1). ¿Qué va a pasar? Escribilo **antes**, en «Tu predicción» del lab.

Primer control · 10–15 min: 3 respuestas con tokens.



Cuatro propiedades explican qué falla

1 Predice el próximo token

escribe lo probable, pedazo a pedazo

Le va bien
Resumir

Le cuesta
Datos precisos

2 Sabe lo que vio al entrenar

conocimiento: nada de después ni de tu empresa

Le va bien
Lo frecuente

Le cuesta
Lo reciente o local

3 Lee lo que le pasás

memoria de trabajo: tu pedido, hasta un máximo

Le va bien
Texto corto

Le cuesta
Un dato en el medio de uno largo

4 Sigue instrucciones

lo entrenaron con pedidos y respuestas

Le va bien
Formato

Le cuesta
Cadenas largas de pasos

Cuando algo falla, preguntate cuál de las cuatro fue.

Cerrados por API, abiertos en tu máquina

	Cerrados, por API	Abiertos, en tu máquina
Dónde corre	En el proveedor	En tu computadora
Tus datos	Salen	Quedan
Qué se publica	Ni pesos ni datos	Los pesos, casi nunca los datos

Muchas empresas hacen modelos de lenguaje

- **Gemini (Google)**
- **GPT (OpenAI)**
- **Claude (Anthropic)**
- **Llama (Meta)**
- **Mistral**
- **Qwen (Alibaba)**
- **DeepSeek**
- **Ollama, en tu máquina**

Usamos Gemini por el plan gratis. Lo que aprendés sirve para todos.

Repaso: hoy te llevás tres ideas



Predice

elige el próximo token
por su probabilidad



Temperatura

cambia la variedad, no
la verdad



Inventa

dice lo probable.
Chequeá nombres, fechas
y cifras.

Antes de irte: ¿qué te sorprendió?

¿Qué viste que contradijo tu predicción? ¿Qué lo explicaría?

Anotá tu **confianza (1 a 4)** y el tema que pensás usar.

El miércoles no hay que leer nada antes: arrancamos con prompts.